Отчёт: AI-агент распределения заказов

Отчёт по тестовому заданию: AI-агент распределения заказов между логистами

1. Что сделано

Разработан AI-агент для GigaCowork, который распределяет новые упаковочные листы (ФУЛ) между логистами и готовит строки для реестра Cloud строго по его форме (12 колонок). Решение о группе и логисте принимает детерминированный скрипт на Python; модель только пересказывает результат.

Репозиторий: https://github.com/89362468222mail-cloud/order-distribution-agent

2. Инструмент и модель

- **Инструмент:** Python 3.12.3 в WSL Ubuntu-24.04 (Linux).
- **Модель:** GigaChat API **не использовался** (нет ключа). Запуск — прямой вызов детерминированного скрипта `agent/scripts/distribute.py`. В GigaCowork тот же скрипт запускается навыком, модель пересказывает вывод по шаблону из `agent/SKILL.md`.
- **Внешние модели в пайплайне не участвуют** — это осознанное решение ради воспроизводимости (3 прогона обязаны совпадать).
- **Сеть и коннекторы не используются.** Только приложенные файлы.

3. Вопросы заказчику

Полный список — в `docs/questions.md` (12 вопросов Q1–Q12). По каждому: кому адресован, зачем, какое допущение принято до ответа. Ключевые:

1. Как отличать повторную отправку от нового заказа — считаем ли заказ «есть», если он в статусе confirm, с другим поставщиком, если номер ФУЛ не совпадает.
2. Дубль внутри одной пачки — какой ФУЛ основной.
3. Что писать в комментарий повторной отправки и куда.
4. Как нормализовать страну и способ доставки.
5. Что делать, если способ доставки или страна не заполнены.
6. «Чаще всех работает с поставщиком» — по каким заказам, за какой период.

7. Учитывается ли история недоступного логиста.
8. «Наименее загруженный» — статично или накопительно по пачке.
9. Тай-брейки при ничьих.
10. Какие состояния и должности исключаем.
11. «Последняя запись в Cloud» и что делать с пропусками.
12. Формат дат и пустые даты в форме Cloud.

4. Алгоритм

Полное описание — в `docs/algorithm.md` (псевдокод, таблица решений, разбор всех сложных случаев). Кратко:

1. Чтение трёх xlsx, поиск колонок по заголовкам с синонимами.
2. Нормализация: trim + lower + ё→е + схлопывание пробелов; поставщик — ещё удаление знаков препинания.
3. Классификация: новый / повтор в Cloud / дубль в пачке / требует внимания.
4. Группа: air → авиа; не-air + Китай → ВЭД; не-air + не Китай → мультимодальные. Если способ или страна не определены — «не определено».
5. Доступность: только «работает», должность из трёх групп. Ассистенты, руководители, отпускники, уволенные — исключены.
6. Выбор логиста:
   - один поставщик в пачке — один логист (если доступен);
   - иначе чаще всех работал с поставщиком (по всем строкам Cloud);
   - иначе наименее загруженный в группе (загрузка накопительная по пачке);
   - ничьи — по возрастанию табельного номера, затем ФИО.
7. Формирование xlsx с 5 листами.
8. Где данных нет — пишется «не определено».

5. Принятые допущения

Полный список — в `docs/assumptions.md` (A1–A12 + T1–T10). Ключевые:

- A1: повтор = заказ есть в Cloud в любой строке, в любом статусе.
- A2: дубль в пачке — основной ФУЛ с самой ранней датой.
- A4: Китай = {китай, china, cn, hong kong, гонконг, hk}.
- A6: история по поставщику — по всем строкам Cloud, статус не фильтруется.
- A8: загрузка накопительная в рамках пачки.
- A9: тай-брейки — по возрастанию табельного номера, затем ФИО.
- A10: доступен только «работает» + должность из трёх групп.
- A11: граница Cloud — максимум по дате получения УЛ, затем по дате подтверждения, затем по cut-off.
- A12: даты в формате ДД.ММ.ГГГГ; пустая дата — пустая ячейка.

6. Тестовые данные и эталон

Полное описание — в `docs/metrics.md`. Кратко:

- 25 ФУЛ, 33 строки Cloud, 11 сотрудников, эталон по 25 ФУЛ.
- Генератор `data/generator.py` с фиксированным seed — данные воспроизводимы.
- Покрыты все ловушки из ТЗ:
  - повтор в Cloud (FUL-005, FUL-021);
  - дубль в пачке (FUL-009);
  - новый поставщик (FUL-007, FUL-024);
  - предпочтительный логист в отпуске (FUL-014 — Сидоров П.);
  - уволенный логист (FUL-015 — Орлов Д.);
  - вся ММ-группа недоступна (FUL-004, 013, 022, 023);
  - нет номера заказа (FUL-010);
  - нет способа доставки (FUL-011);
  - нет страны, не-air (FUL-012);
  - дата ФУЛ раньше последней записи Cloud (FUL-016);
  - поставщик записан по-разному: «ООО Ромашка», «ООО Ромашка» (двойной пробел), «ООО Ромашка.» (FUL-017, FUL-018).

Для каждой ловушки в эталоне зафиксировано, что агент обязан сделать: не создавать строку, оставить логиста пустым, написать «не определено».

7. Результаты прогонов и метрики

Три прогона на одних и тех же файлах. Полные данные — в `docs/metrics.md`.

Метрика	Значение	Порог по ТЗ
Полное совпадение по ФУЛ	25/25 = **100%**	≥ 95%
Точность по полю «Заказ»	100%	—
Точность по полю «Статус»	100%	—
Точность по полю «Группа»	100%	—
Выдуманных значений	**0**	0
Колонок в «Новые строки Cloud»	**12**	12
Воспроизводимость (3 прогона)	**ДА**	3/3
Время ответа на пачку	0.24–0.25 с	< 60 с

Скриншот/лог одного прогона — см. раздел «Приложение» ниже.

8. Что бы доработал дальше

1. **Поддержка других форматов номеров заказов** (сейчас только `S\d{7}`). Если у заказчика есть номера вида `S-12345` или `S012345` — регулярку надо расширять по факту.
2. **Более гибкие правила группы.** Если заказчик захочет учитывать не только страну отправления, но и страну назначения, бренд, тип груза — правила вынесу в YAML-справочник, чтобы не править код.
3. **Конфигурируемые тай-брейки.** Сейчас — по возрастанию табельного номера. Может понадобиться «по наименьшей загрузке за последние N дней».
4. **Логирование в структурированном виде** (JSON) — для интеграции в корпоративный мониторинг.
5. **Юнит-тесты на правила** (сейчас проверка через прогон и

сравнение с эталоном). Добавил бы `pytest` с покрытием каждой ветки `pick_logist`, `detect_group`, `extract_order`.

6. **Обработка нестандартных листов** (несколько листов в книге, объединённые ячейки). Сейчас читается первый лист.
7. **Объяснение в чат по каждому заказу** — сейчас короткий отчёт, подробности в файле. Можно добавить вывод топа N «требует внимания» прямо в чат.

9. Ограничения

- Номер заказа ищется регуляркой `S\d{7}` в комментарии ФУЛ.
- Граница «последней записи Cloud» — по максимальной дате получения УЛ, затем по дате подтверждения, затем по cut-off.
- Один поставщик в пачке — один логист, если он доступен. Если недоступен — новый логист, следующий заказ этого поставщика пойдёт к нему же.
- Скрипт не пишет в Cloud и 1С, только формирует xlsx.

Приложение: лог одного прогона

```
$ python3 agent/scripts/distribute.py \
    --ful data/ful_1c.xlsx \
    --cloud data/cloud_registry.xlsx \
    --employees data/employees_1c.xlsx \
    --out runs/output.xlsx

Чтение ФУЛ: data/ful_1c.xlsx
  строк: 25, найдено колонок: 10/10
Чтение Cloud: data/cloud_registry.xlsx
  строк: 33, найдено колонок: 12/12
Чтение сотрудников: data/employees_1c.xlsx
  строк: 11, найдено колонок: 4/4
Распределение...
Запись результата: runs/output.xlsx

Новых строк: 15
Повторных отправок: 3
Возможных пропусков: 1
Требует внимания: 7
Файл результата: runs/output.xlsx
```

Полный журнал решений — в файле `runs/run_1.xlsx`, лист «Журнал решений».

Приложение: структура репозитория

```
order-distribution-agent/
├── agent/              — то, что грузится в GigaCowork
│   ├── SKILL.md
│   ├── SYSTEM_PROMPT.txt
│   ├── INSTALL.md
│   ├── scripts/        — Python 3 (stdlib + openpyxl + xlrd)
│   └── references/     — правила и шаблон Cloud
├── data/               — генератор и синтетика
├── tests/              — 3 прогона + метрики
├── runs/               — результаты прогонов
```

```
├── docs/              — вопросы, алгоритм, допущения, отчёт
└── dist/              — финальные архивы
```